

The Agent-Coordinator Pattern

It's form, not content.

Ethan [last name] | TTQA

May 6, 2026

We've built the pieces. How do they work together?

Type	What it does	You've used it
Instructions	Behavioral rules, auto-fire on matching files	Weeks 3-4
Agents	Specialized roles with constrained tools	Weeks 5-6
Skills	On-demand domain knowledge bundles	Week 7
Prompts	Opt-in multi-step workflows	Week 8
Hooks	Deterministic enforcement	Today

No static playbook survives this landscape. The tools change. The needs change. What works is knowing what good looks like. That's a quality skill. **That's your guiding light.**

What does "good" look like when you've never built it before?

I needed to build something I'd never built before. Not just the tools -- I didn't know what "good" even looked like for this domain.

But I knew how to find out. **Stack Overflow. Reddit threads. Official docs.** Go where the experts are. Learn what they consider quality.

Then write that down and give it to the agent.

ConsensusScout (skill)



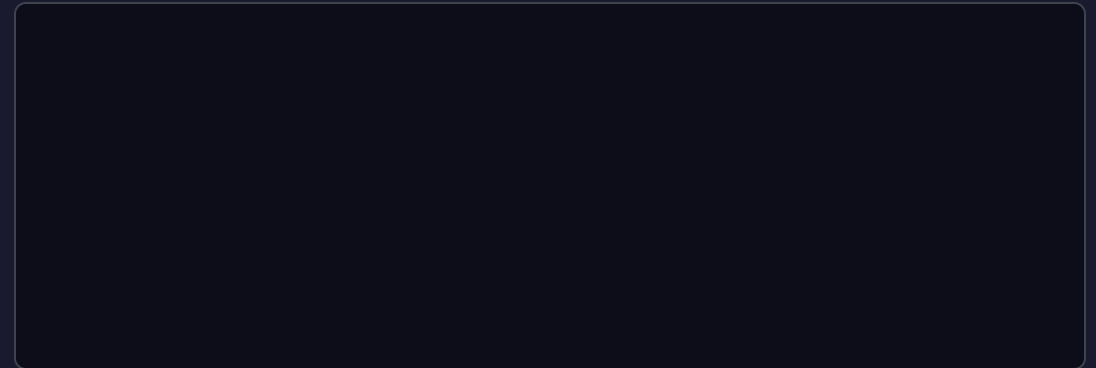
How do you make sure it plans the way you would?

What makes a good instruction file isn't knowing how to "speak agent." It's having opinions on quality and communicating what that looks like.

That means:

- **Discovery** -- knowing what you don't know
- **Requirements** -- defining what done looks like
- **Test plans** -- knowing how to verify it
- **Defect tracking** -- catching drift systematically

code-quality.instructions.md



The core behaviors required to succeed with agents are core behaviors of quality specialists like everyone here.

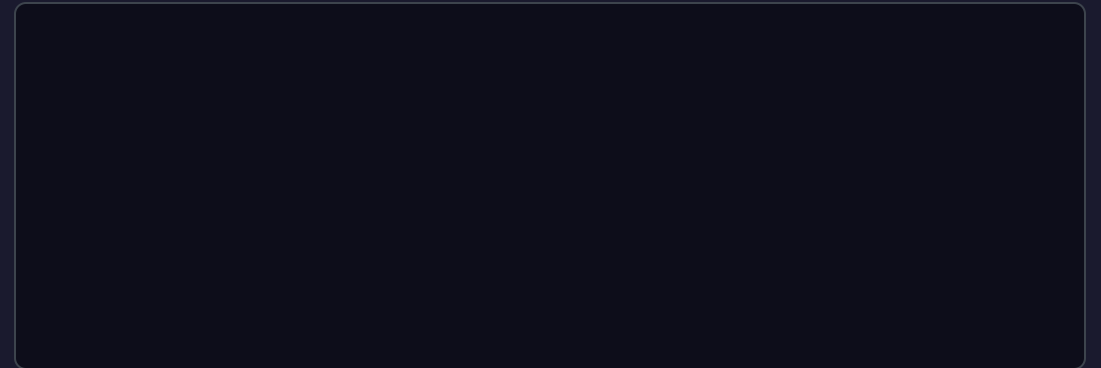
How do you catch what the plan missed?

You've seen AdversarialReviewer. Now you're seeing where it sits: **after the plan forms, before execution starts.**

It pulls holes. Finds what you missed. The agent that makes the other agents better.

Not confident in what came back? Send them back for another round. Have two reviewers argue. Use it as signal -- learn through the cases they make and the citations they reference, then make your own call.

AdversarialReviewer.agent.md



How do you let the agent loose?

I've got decent scaffolding. Instructions, agents, skills, prompts. But I'm invoking each bit manually. The agent doesn't reach for them consistently enough. I forget which prompts I made for what.

That's where **clean lanes** come in -- clear ownership, one job per artifact, organized so the agent finds the right tool at the right time.

And that's where **bypass-permissions** enters. Full agent autonomy. No approval prompts. Just... go.

| Unleashed agent. Sounds terrifying, right?

Where do you draw the line?

Instructions are guidance. The agent *should* follow them. Hooks are enforcement. The agent **can't** do the thing you blocked.

Event	When it fires
PreToolUse	Before the agent runs a tool
PostToolUse	After a tool completes
SessionStart	Agent session begins
UserPromptSubmit	Before your prompt is sent
PreCompact	Before context compression
SubagentStart / Stop	Subagent lifecycle
Stop	Agent session ends

`/create-hook` -- tell the agent what you want to prevent. It writes the hook for

The agent was trying to help.

You've done the quality definition part. Instructions, skills, clear lanes. You've given the agent permissions. But quality definitions are **guidelines, not laws**. How do you put up guardrails to actually stop things you definitely don't want?

I set up a home media server with an agent's help. It needs access to secrets and passwords, so I gave it access to my Vault. It started printing every secret to chat. I'd add a credential and it would helpfully confirm: **"Yep, confirmed it's there: p4\$\$w0rd123 ."**

I rotated so many damn secrets.

That's what hooks are for. The difference between telling an agent not to do something and **making it impossible**.

What did this session teach you?

Review: does the deliverable meet the bar you set? Same instinct you've always had.

Retrospective: what did the session itself reveal?

- What did the agent get wrong that your scaffolding should have caught?
- What hooks do you need that don't exist yet?
- What quality opinions weren't encoded?

Every **"I wish my agent would..."** becomes the next piece of scaffolding. This is the phase that makes everything else compound.

Define quality. Put up guardrails. Iterate.

Research

Quality: ConsensusScout (skill)

Guardrail: source-quality rules (instruction)

Planning

Quality: code-quality principles (instruction)

Guardrail: AdversarialReviewer (agent)

Execution

Quality: working-style constraints (instruction)

Guardrail: block-secrets hook (hook)

Retrospective

Quality: improveInstructions (prompt)

Guardrail: InstructionMaintainer (agent)

The cycle repeats. Each pass, stronger. You've been doing the first half your entire career.

This deck was built using the pattern it describes.

It started with one quality definition: what does good research look like?

Everything else bootstrapped from there.

You don't need to be the expert. You need to know where to find expertise and how to evaluate it.

You're not outsourcing your thoughts to AI.

You're encoding your expertise in a form agents can align with.

| It's not "agent-speak." It's your human-readable prose, just organized.

Get started

Repo: github.com/mydogsburneraccount/agent-scaffolding

Onboarding helpers:

`/onboardProject` | `/generateProjectInstructions` | `/recommendScaffolding`

Contact: [Ethan's preferred contact info]

"How do I start?" Pick one frustration. Turn it into an instruction file.
That's your first piece of scaffolding. It compounds from there.